As with traditional assertion frameworks, the expected value goes first. If this value is a matcher, RSpec will treat it as one; otherwise, it will fall back on comparison using ==.

Here's the full match block now:

12-creating-custom-matchers/09/custom_matcher/spec/support/matchers.rb

```
match { |account| values_match?(amount, account_balance(account)) }
```

The DSL methods you've seen here are all you need to get started with your own matchers. However, RSpec provides several other methods to help you fine-tune your matchers' behavior. You can print meaningful diffs between the actual and expected values, provide custom exception handling, and more. For more information, see the list of DSL methods.[5]

Defining a Matcher Class

Most of the time, you'll use the DSL for defining custom matchers. Sometimes, though, you need a little more control or might prefer to define the matcher in the most explicit way possible.

As we mentioned earlier in the chapter, any Ruby object that implements the matcher protocol can serve as an RSpec matcher. It's pretty easy to translate the DSL example from the previous section into a Ruby class that has the required methods.

The Ruby Class

This class is about a page of code, but it has a lot of similarities to the shorter DSL version we wrote. Take a look here, and then we'll call out a few highlights afterward:

12-creating-custom-matchers/10/custom_matcher/spec/support/matchers.rb

```
Line 1  class HaveABalanceOf
        include RSpec::Matchers::Composable

        def initialize(amount)
          @amount = amount
        end

        def as_of(date)
          @as_of_date = date
          self
        end
```

5. http://rspec.info/documentation/3.6/rspec-expectations/RSpec/Matchers/DSL/Macros.html